

INFO0902-1 : Projet 1 - Algorithmes de sélection

Romain Malaise, Baptiste Renard

24 mars 2025

1 Analyse théorique

1.1 Table des complexités en temps et en espace

Complexité	Pire cas (temps)	Meilleur cas (temps)	Pire cas (espace)	Meilleur cas (espace)
SelectionSelect	$\Theta(k * (n - k))$	$\Theta(k * (n - k))$	$\Theta(1)$	$\Theta(1)$
HeapSelect	$\Theta(k * \log(n) + n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
QuickSelect	$\Theta(n^2)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
FRSelect	$\Theta(n^2)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$

SelectionSelect

Tout d'abord, que ce soit dans le pire ou le meilleur cas, la complexité ne diffère pas. La boucle interne est répétée k fois avec un nombre d'itérations décroissant. Pour la première itération de la boucle externe, la boucle interne itère n fois, ensuite $n - 1$ fois et ainsi de suite jusqu'à $n - k$. Ainsi le nombre d'itérations de la boucle interne est $\sum_{i=0}^n n - i$ qui est de complexité $\Theta(k * n - k^2)$. Nous avons deux implémentations, la première trie le tableau par la gauche en recherchant les plus petites valeurs, le deuxième trie le tableau par la droite en recherchant les plus grandes valeurs. Ainsi, si k est plus grand que $n/2$ alors nous utilisons la deuxième implémentation qui trouvera l'élément k plus rapidement (bien que cela ne change pas la complexité à proprement dit).

En ce qui concerne la complexité en espace, elle est de $\Theta(1)$ dans le pire et meilleur cas, en effet, l'algorithme est toujours en place.

HeapSelect

Dans le cas du HeapSelect, la pire cas est $\Theta(k * \log(n) + n)$ car, nous avons build-max-heap en $\Theta(n)$. La boucle faisant les extractions dépendant de k , en $\Theta(k)$ et max-heapify en $\Theta(\log(n))$ (dépendant de la hauteur de l'arbre binaire complet). Max-heapify étant dans la boucle des extractions, on a $\Theta(k * \log(n))$. Donc au total, on a bien $\Theta(k * \log(n) + n)$.

Pour le meilleur cas est $\Theta(n)$. En effet, le meilleur cas se produit lorsque tous les éléments du tableau sont identiques, et donc par conséquent, Max_heapify ne fait pas d'appels récurrents et est de complexité $\Theta(1)$. Donc nous avons, Build-max-heap de complexité $\Theta(n)$ puis une boucle for qui boucle un certain nombre de fois en fonction de k . Dans cette boucle, Max-heapify est $\Theta(1)$. Donc au final, notre complexité est $\Theta(n + k)$ soit $\Theta(n)$, comme k est borné par n . Notons que la boucle for boucle $n/2$ fois au maximum dans le cas où tous les éléments sont identiques car en fonction de k , nous choisissons de faire un tas min ou un tas max, ce qui va nous permettre de nous arrêter après maximum $n/2$ itérations (ce qui ne change pas la complexité mais nous permet de faire moins d'itérations).

Pour ce qui est de la complexité en espace, c'est évident, l'algorithme est en place et donc la complexité en espace est $\Theta(1)$, que ce soit dans le meilleur ou le pire cas.

QuickSelect

Le pire des cas se trouve quand le pivot ne réduit le tableau de qu'un élément à chaque partitionnement et qu'il trie tous les éléments à gauche et à droite de k avant de finalement arriver à k . Ceci revient à trier tout le tableau comme un quicksort, ce qui est de complexité $\Theta(n^2)$.

Le meilleur des cas, est plus simple à évaluer, il se produit lorsque le pivot tombe directement sur l'élément k que l'on cherche. Ainsi, il suffit de partitionner une fois. Au final la complexité est de $\Theta(n)$.

En ce qui concerne la complexité en espace, elle est toujours $\Theta(1)$, car comme dans nos autres algorithmes, il est toujours en place.

FRSelect

Le pire cas se produit comme dans le pire cas du QuickSelect (bien que cela soit extrêmement peu probable dû à un choix du pivot efficace), c'est-à-dire lorsque le pivot ne réduit le tableau de qu'un élément à chaque partitionnement et qu'il trie tous les éléments à gauche et à droite de k avant de finalement arriver à k . Ceci revient à trier tout le tableau comme un quicksort, ce qui est de complexité $\Theta(n^2)$.

Pour le meilleur cas, il a été montré que¹ la complexité reste $\Theta(n)$ (comme dans son cas moyen d'ailleurs).

En ce qui concerne la complexité en espace, elle est toujours $\Theta(1)$, car comme dans nos autres algorithmes, il est toujours en place.

1.2 Complexité moyenne du QuickSelect

Dans cette section, nous allons montrer que la complexité moyenne du QuickSelect est $\Theta(N)$. Nous avons :

- Nombre de comparaisons pour le partitionnement : $n - 1$
- Probabilité que le pivot soit à la position q : $1/n$
- Tailles des sous-tableaux dans ce cas-là : $q - 1$ et $n - q$. Par contre, nous allons seulement parcourir celui de gauche, soit celui de taille $q - 1$.
- Les sous-tableaux sont aussi triés aléatoirement

Ainsi, le nombre moyen de comparaisons est donné par la récurrence suivante :

$$C_0 = 0$$

$$C_n = n - 1 + \sum_{q=1}^n \frac{1}{n} C_{q-1}, \quad \text{si } n > 0$$

où C_{q-1} est la récurrence pour le tableau à gauche du pivot comme nous savons que l'on cherche le minimum (qui se trouve forcément à gauche du pivot).

$$C_n = n - 1 + \frac{1}{n} \sum_{q=1}^n C_{q-1}$$

1. <https://arxiv.org/abs/cs/0402005>

$$n * C_n = n * (n - 1) + \sum_{q=1}^n C_{q-1}$$

En soustrayant la même formule pour $n - 1$:

$$n * C_n - (n - 1) * C_{n-1} = 2 * (n - 1) + C_{n-1}$$

$$n * C_n = 2 * (n - 1) + n * C_{n-1}$$

$$C_n = 2 - \frac{1}{n} + C_{n-1}$$

$$C_n = 2 - \frac{1}{n} + 2 - \frac{1}{n-1} + C_{n-2}$$

$$C_n = C_{n-i} + \sum_{j=0}^{i-1} (2 - \frac{1}{n-j})$$

Pour $i = n$, nous avons :

$$\begin{aligned} C_n &= C_0 + \sum_{j=0}^{n-1} (2 - \frac{1}{n-j}) \\ &= 2 * n - \sum_{j=0}^{n-1} \frac{1}{n-j} \geq n \in \Theta(n) \end{aligned}$$

Vu que le second terme est borné par n , la différence des deux termes est plus grande que n et plus petite que $2n$

1.3 Stabilité

SelectionSelect

Cet algorithme de sélection n'est pas stable. Le SelectionSelect s'arrête au milieu de tri lorsqu'il a trouvé k , ce qui ne garantit pas que l'ordre des éléments égaux seront identiques à celui d'avant le tri. Considérons l'exemple suivant :

Nous avons



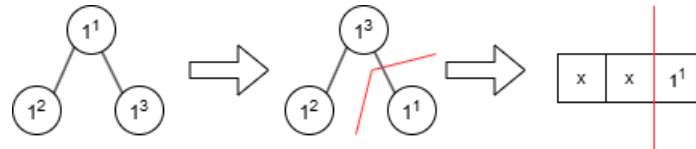
A noter que si nous avions recherché un k plus grand que $n/2$, nous aurions trié par la droite en cherchant les maximums, ce qui n'aurait pas changé notre problème de stabilité.

HeapSelect

Cet algorithme n'est pas stable. En effet, nous pouvons trouver un contre-exemple assez facilement. Prenons le tableau suivant :

1^1	1^2	1^3
-------	-------	-------

Dans le cas du HeapSelect avec tas max, l'élément 1^1 sera d'office mis à la fin du tableau, ce qui ne garantit pas la stabilité.



QuickSelect

Ici, l'algorithme n'est pas stable non plus, en effet, on peut trouver un contre-exemple. Prenons le tableau suivant et simulons une partie de l'algorithme dessus. Le 3 qui se trouve à la fin du tableau ne bougera plus, en effet si nous prenons k égal à 0, nous ne visiterons plus la partie à droite de 2 dans le tableau, ce qui laissera l'ordre des deux 3 inversé.



FRSelect

Étant donné que le Floyd-Rivest select se base sur le principe du QuickSelect, il hérite du même problème de stabilité.

2 Expérimentations

2.1 Table des temps de calcul

Type de tableau	croissant			décroissant		
Taille	10^4	10^5	10^6	10^4	10^5	10^6
SelectionSelect	72 ms - 37502499	7890 ms - 375002499	/	81 ms - 37502499	9160 ms - 375002499	/
HeapSelect	0.7 ms - 128072	7.6 ms - 1609819	119 ms - 19421127	0.8 ms - 142354	10 ms - 1757001	139 ms - 20865889
QuickSelect	0.08 ms - 29993	0.678 ms - 286492	5.4 ms - 2144984	0.04 ms - 15000	0.43 ms - 150000	5.6 ms - 1500001
FRSelect	0.017 ms - 10428	0.181 ms - 109937	2.37 ms - 1345442	0.059 ms - 30041	0.625 ms - 266398	6.7 ms - 2740024

TABLE 1 – Résultats expérimentaux pour $k = n/2$

Type de tableau	croissant			décroissant		
Taille	10^4	10^5	10^6	10^4	10^5	10^6
SelectionSelect	18.18 ms - 9508499	2044 ms - 960084999	/	19.68 ms - 9508499	2683 ms - 95008499	/
HeapSelect	0.14 ms - 34361	1.72 ms - 411263	22.68 ms - 4777979	0.19 ms - 45732	2.37 ms - 517581	36.32 ms - 5782975
QuickSelect	0.086 ms - 29997	0.75 ms - 299996	8.1 ms - 2999947	0.15 ms - 57798	1.9 ms - 579697	22.7 ms - 5799377
FRSelect	0.016 ms - 10300	0.19 ms - 113793	2.58 ms - 1422985	0.05 ms - 34061	0.44 ms - 216427	4.71 ms - 2277556

TABLE 2 – Résultats expérimentaux pour $k = n/10$

Type de tableau	aléatoire			constant		
Taille	10^4	10^5	10^6	10^4	10^5	10^6
SelectionSelect	75 ms - 37502499	8280 ms - 375002499	/	75 ms - 37502499	8740 ms - 375002499	/
HeapSelect	0.78 ms - 128051	10 ms - 1704243	168 ms - 20315726	0.08 ms - 19999	0.8 ms - 199999	6.61 ms - 1999999
QuickSelect	0.17 ms - 40350	1.84 ms - 422315	22.4 ms - 4017101	0.05 ms - 20001	0.53 ms - 200001	5.3 ms - 2000001
FRSelect	0.08 ms - 21110	1.03 ms - 177278	9.12 ms - 1627822	0.07 ms - 21110	0.80 ms - 205656	7.78 ms - 2027728

TABLE 3 – Résultats expérimentaux pour $k = n/2$

Type de tableau	aléatoire			constant		
Taille	10^4	10^5	10^6	10^4	10^5	10^6
SelectionSelect	18.15 ms - 9508499	2039 ms - 95008499	/	19.74 ms - 9508499	2039 ms - 95008499	/
HeapSelect	0.26 ms - 43259	3.19 ms - 499226	46.39 ms - 5645319	0.03 ms - 11999	0.33 ms - 119999	4.25 ms - 1199999
QuickSelect	0.013 ms - 32496	1.73 ms - 348793	16.8 ms - 3395778	0.05 ms - 20001	0.54 ms - 200001	5.3 ms - 2000001
FRSelect	0.06 ms - 21176	0.79 ms - 205954	7.62 ms - 2028562	0.06 ms - 21176	0.79 ms - 205954	7.62 ms - 2028562

TABLE 4 – Résultats expérimentaux pour $k = n/10$

2.2 Commentaires

2.1 SelectionSelect

Nous remarquons immédiatement que le nombre de comparaisons dépend uniquement de n et k . En effet, nous retrouvons pour un n et un k donné exactement le même nombre de comparaisons. Étant donné notre complexité calculée, le nombre de comparaisons devrait grandir de façon quadratique et c'est ce qu'on a. Par exemple, nous avons pour le type de tableau croissant de taille de 10^4 et 10^5 respectivement 72ms et 7890 ms pour $k = n/2$ (idem pour $k = n/10$). Donc, cela vérifie la croissance quadratique. Ceci est expliqué par le fait que le SelectionSelect ne possède pas de pire et meilleur cas. Autrement dit, qu'il s'exécute sur un tableau trié de façon aléatoire, croissante, décroissante ou constante, il fera la même chose.

En ce qui concerne le paramètre k , plus k s'éloigne de $k/2$, plus le k -ième élément est trouvé rapidement. Au contraire, plus k est proche du centre (c'est-à-dire $k/2$), plus il prend du temps à trouver. (Ce qu'on observe en jettant un simple coup d'oeil dans les résultats expérimentaux). Ceci s'explique simplement par notre implémentation. Nous avons implémenté une spécificité

qui fait en sorte que si k est plus proche de la fin du tableau, nous recherchons la position k par les maximums. Tandis que si k est plus proche du début du tableau, nous cherchons par les minimums. Ce qui conduit à une exécution plus rapide lorsque k est proche des extrémités.

2.2 HeapSelect

Nous observons que le nombre de comparaisons est presque linéaire par rapport à n et k . Cependant nous remarquons quand même une légère augmentation en plus. On peut remarquer une grande différence entre $k = n/2$ et $k = n/10$, ce qui est logique. En effet, pour $k = n/2$, HeapSelect doit trier le tableau à moitié alors que pour $k = n/10$, HeapSelect ne doit trier qu'un dixième du tableau. De plus nous avons implémenté une optimisation, si k est plus proche de la fin du tableau, nous utilisons un tas-max plutôt qu'un tas-min, car nous trouvons k plus rapidement par les grandes valeurs. Pour finir, nous pouvons remarquer que pour le HeapSelect, le cas d'un tableau constant rend l'algorithme très rapide et correspond à notre complexité du meilleur cas $\Theta(n)$. En ce qui concerne l'explication, elle a déjà été démontrée dans la section 1.1.

2.3 QuickSelect

Nous retrouvons une complexité linéaire. En effet, si n et k sont multipliés par 10 alors le temps et le nombre de comparaisons aussi. Une deuxième observation est que les résultats des tableaux constants sont toujours les meilleurs. Ceci s'explique par notre optimisation de la partition en trois parties. Nous partitionnons notre tableau en trois parties : strictement inférieure, égale et strictement supérieure, ainsi, si la position k fait partie de la partie 'égale' alors nous avons directement fini notre recherche. Cette optimisation réduit le nombre d'appels récurifs nécessaires, et, dans le cas d'un tableau constant, trouve la solution en une seule partition. Pour finir, nous pouvons mentionner le choix du pivot par la médiane de trois valeurs : la première, la dernière et celle du milieu du tableau. Ainsi nous trouvons une valeur qui ressemble à la vraie médiane du tableau (le meilleur choix de pivot pour le QuickSelect).

2.4 FRSelect

Tout d'abord, rappelons nous que le FRSelect est une variante du QuickSelect, sauf que celui-ci choisit mieux les pivots et est donc plus performant dans tous les cas. De plus, cette affirmation semble à première vue bien correcte, que ce soit au niveau des comparaisons ou des temps, on observe que le FRSelect est meilleur. Pour un $k = n/10$, il est légèrement meilleur que pour $k = n/2$.

2.3 Commentaire sur l'ordre relatif des algorithmes

Le FRSelect est l'algorithme le plus rapide et celui qui fait le moins de comparaisons de tous les algorithmes présentés, dans la quasi-totalité des cas. Ensuite, le QuickSelect, est aussi très rapide dans toutes les situations, notamment grâce aux spécificités des implémentations (médiane et partitionnement en trois parties). Cependant, le FRSelect semble bien meilleur que le QuickSelect lorsque le k tend à être extrême. Si l'on souhaite résoudre le problème pour un k extrême, nous avons d'autant plus l'intérêt à utiliser le FRSelect, qui choisira mieux le pivot en fonction de k et n , pour être plus efficace dans ces cas là. Nous observons aussi que le FRSelect, par le moyen d'un bon choix de pivot, fait moins de comparaisons que le QuickSelect. Ensuite, après les deux algorithmes, en terme de rapidité et du nombre de comparaisons, il y a le HeapSelect et puis le SelectionSelect, qui lui est inutilisable lorsque la taille du tableau devient de l'ordre de 10^5 , 10^6 .